

Complete Codebase Review — Go Trading Bot

Stakes: ~\$500M USD in capital under management **Reviewed:** All Go source files, migrations, config, .env, and docs

Foreword

This review is written assuming this system will be used to execute real orders on NSE using Zerodha Kite Connect. At \$500M scale, a single unchecked bug in the execution path is not a code quality problem — it is a financial catastrophe. The review is structured in descending order of severity. Read Section 1 before touching anything else.

Section 1 — SHOWSTOPPER: The System Cannot Trade

This is the most critical finding in the entire review. **There is no order execution code.** Not incomplete order execution — literally none.

`internal/execution/order.go` contains two lines:

```
go

// Package monitor provides system monitoring and order execution utilities.
// Order placement logic is not yet implemented.
```

`internal/broker/doc.go`, `internal/services/doc.go`, `internal/strategy/intraday.go`,
`internal/strategy/scalping.go`, `internal/strategy/swing.go` — all empty stubs.

The system today is a sophisticated **market data observation platform**, not a trading bot. It ingests ticks, builds candles, calculates 10 indicators, and streams them via WebSocket. But it has no code that:

- Generates a trade signal from indicator values
- Places an order through Kite Connect
- Manages open positions
- Applies a stop-loss or take-profit
- Tracks P&L

The `ZerodhaClientInterface` does not include `PlaceOrder`, `CancelOrder`, or `GetPositions`. The `kiteconnect` SDK provides all of these, but they are not called anywhere.

Before \$500M is at risk, the full execution path — signal → risk check → order → position management — must be built, audited, and paper-traded for weeks. The data pipeline reviewed here is solid groundwork, but it is not a trading system.

Section 2 — CRITICAL: Active Runtime Bombs

These are bugs that will crash or corrupt the live system during a trading day.

2a. `zap.L().Fatal()` inside MACD calculation kills the server mid-session

File: `internal/indicators/indicators.go`, lines 356 and 396

```
go
zap.L().Fatal("MACD calculation fatal error: could not find original candle for MACD")
```

`zap.L().Fatal()` calls `os.Exit(1)`. This means if a MACD timestamp lookup misses — which can happen on a restart when candle history is partially loaded — the entire server process terminates immediately. There is no graceful shutdown, no open-order cancellation, no position close. Whatever positions happen to be open at that moment stay open with no oversight.

Fix: return an error instead of calling Fatal.

2b. `zap.L().Fatal()` on DB flush timeout kills server under load

File: `internal/data/ingest.go`, lines 321 and 375

```
go
zap.L().Fatal("DB flush timeout - cannot persist market data. Exiting.")
```

Under Postgres load spike — entirely normal at market open (9:15 AM) when hundreds of instruments emit ticks simultaneously — the DB flush channel fills and the 5-second timeout fires. This is a **self-inflicted crash during the highest-volume period of the day**.

Fix: increment a dead-letter counter, log at Error, and continue. Losing some ticks to DB is acceptable; losing the process is not.

2c. Unguarded array access on empty market depth — guaranteed panic on illiquid instruments

File: `internal/data/ingest.go`, lines 259-294

```
go
```

```

buyDepth := tick.Depth.Buy    // []DepthItem, may have < 5 elements
sellDepth := tick.Depth.Sell
zap.L().Debug("Tick received",
    zap.Float64("bid", buyDepth[0].Price),    // PANIC if len < 1
    zap.Float64("ask", sellDepth[0].Price),    // PANIC if len < 1
)

```

The code then accesses `buyDepth[0]` through `buyDepth[4]` and `sellDepth[0]` through `sellDepth[4]` unconditionally. The Kite Connect documentation states that depth data is only present in `ModeFull`. For illiquid instruments, option strikes, or any tick received in `ModeLTP` or `ModeQuote`, the depth arrays can be shorter than 5. This is an index-out-of-bounds panic that will crash every goroutine processing that tick.

Fix: check `len(buyDepth) >= 5 && len(sellDepth) >= 5` before accessing.

2d. Nested lock + blocking channel send = guaranteed deadlock

File: `internal/marketdata/candles/candle_engine.go` + `internal/data/candles.go`

`ProcessTick` and `finalizeByWatermark` both hold `e.mu` (the engine's write lock) when they call `finalizeCandleLocked`, which calls `cfg.OnFinalize`, which is `cg.handleFinalizedCandle`. That callback does:

```

go

select {
case cg.candleDBFlushCh <- ohlcv: // BLOCKS if channel is full
default:
    // drops
}

select {
case cg.indicatorManagerInputCh <- indicatorCandle: // BLOCKS if channel is full (5
default:
    // drops
}

```

The `default` branch means both sends are non-blocking, so a drop occurs rather than a deadlock. However `broadcastCandle` is then called, which iterates `candlewsClients` doing non-blocking sends. This is fine.

But there is a subtler problem: `OpenCandle.mu` is locked inside `ProcessTick` while `e.mu` is already held:

```

go

```

```
e.mu.Lock()           // outer lock
...
candle.mu.Lock()      // inner lock – acquired inside outer
candle.mu.Unlock()
```

`finalizeByWatermark` also holds `e.mu` and calls `finalizeCandleLocked` which reads candle fields directly (no `candle.mu` there). If another goroutine holds `candle.mu` and tries to acquire `e.mu`, you have a deadlock. Audit all call sites where `candle.mu` is acquired and confirm `e.mu` is never also required.

2e. Zerodha ticker has no reconnection logic

File: `internal/api/ticker.go`

```
go

z.Ticker.OnClose(func(code int, reason string) {
    zap.L().Warn("Zerodha WebSocket closed", ...)
    // Nothing else. No reconnect.
})
```

When the Kite Connect WebSocket closes mid-session (network blip, Zerodha maintenance), the system logs a warning and stops receiving ticks. `MarketDataIngestor` continues running, `CandleGenerator` continues, but both receive nothing. The system appears alive while it is blind. No reconnect attempt, no alert that data has stopped.

Fix: implement exponential backoff reconnection in `OnClose` using a goroutine, similar to how `subscribeAndProcessRedis` already handles reconnects.

2f. `loadHistoricalCandles` loads the entire `ohlcv_candles` table into memory on every restart

File: `internal/data/indicators_manager.go`

```
go

err := im.dbClient.DB.Raw(`
    SELECT instrument_token, interval, timestamp, open, high, low, close, volume
    FROM ohlcv_candles
    WHERE interval = ?
    ORDER BY instrument_token, timestamp ASC
    `, interval).Scan(&rows).Error
```

There is **no LIMIT clause**. After a month of trading, `ohlcv_candles` will contain millions of rows per interval. On every restart, this query loads all of them. The code then slices to `maxNeeded` per

token in Go memory — after already loading everything from Postgres into RAM. This will cause OOM crashes as the table grows, and will make restarts take minutes at peak data volumes.

Fix: add `LIMIT` per token using a subquery, or use a window function:

```
sql

SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY instrument_token ORDER BY timestamp DE
    FROM ohlcv_candles WHERE interval = ?
) t WHERE rn <= ?
```

Section 3 — CRITICAL: Security — Credentials Committed to Repository

3a. Live API credentials in `.env` committed to the repo

File: `.env` (committed, not gitignored effectively for this review)

```
ZERODHA_API_KEY=n19aa6q80bsefmmm
ZERODHA_API_SECRET=f7aqjn83wlhp01uaaw77qzui6qtii1tl
REDIS_URL=redis://default:TkwkBw17NElpkqCGITprJz3JzUFkxTl6@redis-15547...
JWT_SECRET=e8c56b56d04b21fea938ac853e5fddd248939b0d036260e7bc263a7c7f8eaf45
DB_PASSWORD=admin
```

These are live credentials. If this repository is or has been on a remote (GitHub, GitLab, etc.), these secrets must be rotated **immediately**. The Zerodha API secret alone allows anyone to generate access tokens and place orders on your account.

The `.access_token` file (32 bytes, a live access token) is also committed. Rotate this too.

Additionally, `.gitignore` lists `.env` but `.access_token` is not gitignored. Verify your git history does not contain these values.

3b. Redis URL contains credentials in plaintext

The Redis Cloud URL with password embedded (`TkwkBw17...`) is in the committed `.env`. Rotate it.

3c. `DATA_ENCRYPTION_KEY` is a weak human-readable string

```
DATA_ENCRYPTION_KEY="a_very_strong_32_byte_key_for_encryption"
```

This is not a real key. It appears to be a placeholder that was never replaced. The

`UserBrokerAccount` table stores encrypted access tokens — if this key was used in production, all

broker tokens stored in the DB are encrypted with a known-plaintext key. Regenerate with `openssl rand -base64 32` and re-encrypt all stored tokens.

Section 4 — HIGH: Missing PO Safety Controls

These are controls that every production trading system must have before live execution. They are referenced in the config (`MaxLossPerDayPercentage`, `TradeSizePercentageOfCapital`) but not implemented in any code.

4a. No Global Kill Switch

There is no mechanism to halt all trading activity instantly. At institutional scale, when something goes wrong (erroneous signal, data corruption, external event), a trader needs to press one button and stop everything. The `runtime` package exists but has no `Halt` concept. There is no `/api/v1/runtime/halt` endpoint, no emergency Redis key that workers poll, and no circuit-breaker pattern anywhere.

4b. No Daily Loss Limit Engine

`MaxLossPerDayPercentage` is defined in config but read only into the config struct — it is never checked anywhere in the codebase. There is no code that reads today's realized P&L, compares it to the limit, and refuses to place further orders if the limit is breached.

4c. No Order Idempotency Key

There is an `Order.BrokerOrderID` field in the DB model, but no idempotency enforcement at the placement layer. In a high-frequency or reconnect scenario, the same signal could fire twice and place duplicate orders. The Kite Connect API supports tag-based order tracking for exactly this purpose.

4d. No Rate Limiting on Order Placement

Kite Connect enforces order rate limits. Exceeding them results in HTTP 429, which the SDK will raise as an error. If the strategy logic retries naively on error, it can create an order retry storm.

Section 5 — HIGH: Architecture Issues (The Three Review Points)

5a. Interfaces: `ZerodhaClientInterface` is defined but bypassed in 4 places

`handlers.go` does `client, ok := deps.ZerodhaClient.(*api.ZerodhaClient)` three times. The interface is meaningless if every call site punches through it. All methods actually needed (`GetUserProfile`, `GetQuote`) are already on the interface — remove the type assertions and use the interface methods directly.

5b. Interfaces: No interfaces for `DBClient`, `RedisClient`, or `MarketHeatmap`

All three data pipeline components (`MarketDataIngestor`, `CandleGenerator`, `IndicatorManager`) accept concrete `*db.DBClient` and `*cache.RedisClient` pointers. You cannot unit test any component without a live Postgres and Redis. The `MarketHeatmap` global singleton (`GetMarketHeatmap()`) is called from handlers, Hub, and the ingestor — shared mutable global state makes test isolation impossible.

Define narrow interfaces at each boundary:

```
go

type TickStore interface {
    SaveMarketDataBatch(ctx context.Context, batch []MarketData) error
}

type Cache interface {
    Set(key string, value interface{}, expiration time.Duration) error
    Get(key string) (string, error)
    Publish(channel string, message interface{}) error
    Subscribe(ctx context.Context, channels ...string) *redis.PubSub
}

type HeatmapWriter interface {
    Update(symbol string, ...)
}
```

5c. Coupling: `internal/data` owns four unrelated concerns

`data/ingest.go` (tick pipeline), `data/candles.go` (candle generation), `data/indicators_manager.go` (indicator calculation), and `data/heatmap.go` (global market snapshot) are all in the same package. The `wsClient` struct is defined in `indicators_manager.go` but used in `candles.go` — an accidental shared type. These are four distinct domain services that should live in four separate packages.

5d. Coupling: `ingest.go` imports the Zerodha SDK directly

```
go

kitemodels "github.com/zerodha/gokiteconnect/v4/models"
```

The market data ingestor converts `NormalizedTick` back to `kitemodels.Tick` for downstream processing. This means your entire data pipeline is Zerodha-specific at the type level. The `marketdata.NormalizedTick` type already exists for exactly this abstraction — use it throughout instead of converting back to the broker-specific type.

5e. App Container: `app.go` imports 10 packages and has 3 responsibilities

The `New()` function constructs every component, wires them together, and contains ~80 lines of raw SQL and DB upsert logic for both simulation and live modes. The instrument upsert code belongs in a dedicated `InstrumentSyncService`. `app.go` should be a composition root only.

5f. Triple type assertion in `server.go` to register one route

```
go

if zc, ok := s.deps.ZerodhaClient.(interface{ GetKite() interface{} }); ok && zc != nil {
    if _, ok := s.deps.ZerodhaClient.(interface{ FindInstrumentToken... }); ok {
        if zc, ok2 := s.deps.ZerodhaClient.(*api.ZerodhaClient); ok2 && zc != nil {
```

Three assertions on the same value to register `/api/instrument`. The comment in the code itself says "this is a bit messy." Fix: `HandleInstrumentLookup` should accept `ZerodhaClientInterface`, not the concrete type.

Section 6 — MEDIUM: Data Correctness Bugs

6a. Rate limiter IP extraction trusts `X-Forwarded-For` without validation

File: `internal/middleware/ratelimit.go`

```
go

if xff := r.Header.Get("X-Forwarded-For"); xff != "" {
    return strings.SplitN(xff, ",", 2)[0]
}
```

`X-Forwarded-For` is a client-supplied header. Any user can set it to any value, effectively bypassing per-IP rate limiting by rotating fake IPs. In a trading context, this could allow a brute-force attack on the `/auth/login` endpoint. Only trust this header if you are behind a known reverse proxy and you strip the header before it reaches your application.

6b. Refresh token is blocklisted but the check is wrong

File: `internal/api/handlers/auth/logout.go`

The logout handler blocklists the refresh token with key `blocklist:refresh:` + token. The middleware that checks blocklists (in `auth.go`) checks `blocklist:access:` + token for access tokens. The refresh token check in `refresh.go` checks `blocklist:refresh:` + token. This is consistent. However: the `HandleLogout` function is on the **public** route (not under the `protected` subrouter). A user can call logout without a valid access token. The blocklist is populated based

on parsing the Authorization header, which may not be present. This is acceptable by design but should be explicitly documented.

6c. Candle volume is stored as `float64` but should be integer

File: `internal/db/models.go`, `OHLCVCandle`

```
go
Volume float64 `gorm:"not null"`
```

Exchange volume is always a whole number of shares. Storing as `float64` can cause floating-point accumulation errors when summing volumes in SQL. Use `int64` or `bigint`.

6d. `loadHistoricalCandles` ignores `TradeCount` — indicator history is incomplete

The historical candle loader in `IndicatorManager` does not query `trade_count` from `ohlcv_candles`. The `indicators.Candle` struct has a `TradeCount` field. Indicators that depend on trade count (currently OBV approximations could be affected) will have zero trade counts for all historical candles on restart.

6e. `processedIndicatorCh` drop counter is mislabeled as `wsDrops`

File: `internal/data/indicators_manager.go`

When the indicator output channel is full and a result is dropped, the code increments

`im.wsDrops`:

```
go
default:
    atomic.AddUint64(&im.wsDrops, 1)
```

This is an indicator calculation drop, not a WebSocket drop. The monitoring metric conflates two different failure modes, making it impossible to distinguish "frontend clients are slow" from "indicator pipeline is overloaded."

Section 7 — MEDIUM: Operational and Reliability Gaps

7a. No health check for the ticker connection

`/api/v1/health` checks Postgres and Redis connectivity but not whether the Zerodha ticker is actually receiving ticks. In live mode, if the ticker silently disconnects (see 2e), the health endpoint still returns `ok`. You need a heartbeat: record the last tick received time in Redis and expose it in health. Alert if it is older than 30 seconds during market hours.

7b. The `monitorStopCh` in `CandleGenerator` is never stopped by the `RuntimeManager`

File: `internal/app/app.go`, service registration

The commented-out stop handler for `candle_generator`:

```
go

// stop: func(ctx context.Context) error {
//     return app.CandleGenerator.Stop(ctx)
// },
```

The `Stop()` method exists and is correct (calls `FinalizeAll`, closes monitor channel). It is simply not wired. This means on graceful shutdown, in-progress candles are not flushed and the monitoring goroutine leaks. Uncomment and wire this.

7c. `CandleGenerator` starts its own `StartCandleDBWriter` inside `StartCandleGeneration`, but `RuntimeManager` also registers `candle_db_writer` separately

File: `internal/app/app.go`, `internal/data/candles.go`

```
go

// In app.go:
app.rm.Register(&serviceAdapter{name: "candle_db_writer", start: func(...) {
    go app.CandleGenerator.StartCandleDBWriter(ctx)
}})

// In candles.go, inside StartCandleGeneration:
go cg.StartCandleDBWriter(ctx)
```

`StartCandleDBWriter` is launched **twice** — once explicitly by `RuntimeManager` and once inside `StartCandleGeneration`. Two goroutines are now draining `candleDBFlushCh`, both writing to the same DB. This creates a race for candles: whichever goroutine dequeues a candle first writes it; the other goroutine may then receive the next candle and attempt a conflicting write. The `DoUpdates` clause handles the conflict for known columns, but this is a correctness bug hiding behind database-level conflict resolution.

7d. `startSimulatedFeed` drops FK constraints with `CASCADE` — in-place, no restore

File: `internal/app/app.go`

```
go

"ALTER TABLE market_data DROP CONSTRAINT IF EXISTS fk_market_data_instrument_token C
```

This removes a foreign key with cascade in the live database to allow simulation upserts. There is no code to restore these constraints after simulation ends. If the system is started in simulation mode and then switched to live mode (or the process crashes), the DB runs permanently without FK enforcement. Orphaned market data rows become possible.

7e. Audit log `goroutine` fire-and-forget with no error surfacing

File: `internal/audit/audit.go`

Every `LogEvent` call spawns an anonymous goroutine:

```
go
go func() {
    if err := repo.Log(context.Background(), auditEvent); err != nil {
        logger.Error("Failed to write audit event to DB", zap.Error(err))
    }
}()
```

At 500M scale with high-frequency auth events, this creates a goroutine-per-event pattern. If the DB is slow, thousands of goroutines pile up. Use a buffered channel with a single writer goroutine, or a batch inserter.

Section 8 — MEDIUM: Code Quality

8a. 250-line `switch` statement in `indicators_manager.go` using type switches

`calculateAndStoreAllIndicators` has a 200+ line type switch that handles each indicator type identically (check length, check NaN, send to channel). Every indicator has the same structure. This should be replaced with a uniform post-processing function that accepts an `IndicatorResult` interface.

8b. `handleOutput` in `indicators_manager.go` has a duplicate switch for conflict columns

One switch determines conflict columns; `db/utls.go` has another switch for updatable columns for the same set of indicator names. These are always in sync today but will diverge when a new indicator is added and one switch is updated but not the other. Consolidate to a single indicator registry struct.

8c. `wsClient` struct defined in `indicators_manager.go`, used in `candles.go`

Shared type across files in the same package — an accidental coupling. When these are split into separate packages (as they should be), this will cause a compile error.

8d. `GetUserIDFromContext` in `auth/jwt.go` vs `UserIDFromContext` in `middleware/context.go` — duplicate function, different context keys

`auth.GetUserIDFromContext` reads context key `"userID"` (plain string).

`middleware.UserIDFromContext` reads context key `UserIDKey` (typed `contextKey`). Both exist. If any handler uses the `auth` package version, it will always return 0 because the middleware writes using the typed key. This is a silent bug.

8e. Rate limiter `cleanup` goroutine runs forever with no stop mechanism

File: `internal/middleware/ratelimit.go`

The `cleanup()` goroutine starts with `go r1.cleanup()` and has no way to be stopped. On graceful shutdown, it leaks. Use a stop channel or pass the server context.

Section 9 — LOW: Test Coverage Assessment

Coverage is thin for a financial system:

Package	Test Status
<code>internal/auth</code>	✓ <code>jwt_test.go</code> — reasonable coverage of sign/parse/claims
<code>internal/middleware</code>	✓ <code>middleware_test.go</code> — rate limiter, auth middleware
<code>internal/contracts</code>	✓ <code>contracts_test.go</code>
<code>internal/security</code>	✓ <code>encryption_test.go</code> , <code>redaction_test.go</code>
<code>internal/validation</code>	✓ <code>validation_test.go</code>
<code>internal/marketdata</code>	✓ <code>tick_test.go</code>
<code>internal/data</code>	⚠ <code>data_test.go</code> , <code>websocket_test.go</code> — mostly WebSocket infra
<code>internal/indicators</code>	✗ No tests — critical gap : SMA, EMA, MACD, ATR, RSI, etc. have no correctness tests
<code>internal/data</code> (candles)	✗ No tests for candle bucketing, alignment, or finalization
<code>internal/app</code>	✗ No tests
<code>internal/execution</code>	✗ Empty

Prioritized Remediation Table

Priority	Issue	File	Effort	If Ignored
P0-BLOCK	No order execution code	<code>internal/execution/</code>	Weeks	System cannot trade
P0-CRASH	<code>Fatal()</code> in MACD kills server mid-session	<code>indicators/indicators.go:356,396</code>	1h	Process dead with open positions
P0-CRASH	<code>Fatal()</code> on DB flush timeout	<code>data/ingest.go:321,375</code>	1h	Process dead at market open
P0-PANIC	Unguarded depth array access <code>[0..4]</code>	<code>data/ingest.go:259-294</code>	1h	Panic on illiquid instruments
P0-SEC	Live API key + secret in committed <code>.env</code>	<code>.env</code>	Immediate	Account takeover
P0-SEC	Live access token committed <code>(.access_token)</code>	<code>.access_token</code>	Immediate	Unauthorized trading
P0-SEC	Weak <code>DATA_ENCRYPTION_KEY</code>	<code>.env</code>	2h	All stored broker tokens insecure
P0-BUG	Duplicate <code>StartCandleDBWriter</code> launch	<code>app.go</code> + <code>candles.go</code>	30m	Double-write race condition
P1	No Zerodha WebSocket reconnection	<code>api/ticker.go</code>	3h	Blind system after network blip
P1	<code>loadHistoricalCandles</code> no LIMIT — OOM on restart	<code>data/indicators_manager.go</code>	1h	OOM crash grows over time

Priority	Issue	File	Effort	If Ignored
P1	No Global Kill Switch	<code>internal/runtime/</code>	1 day	No emergency stop
P1	No Daily Loss Limit enforcement	config + new service	1 day	Unlimited loss possible
P1	<code>CandleGenerator.Stop()</code> not wired	<code>app/app.go</code>	15m	In-progress candles lost on shutdown
P1	FK constraints dropped, never restored	<code>app/app.go</code>	2h	DB without referential integrity
P2	<code>ZerodhaClientInterface</code> bypassed by type assertions	<code>httpapi/handlers.go</code>	2h	Interface is decorative
P2	No interfaces for DB/Redis/Heatmap	Multiple	4h	Untestable component
P2	Rate limiter trusts <code>X-Forwarded-For</code>	<code>middleware/ratelimit.go</code>	1h	IP rate limit bypassable
P2	<code>GetUserIDFromContext</code> key mismatch	<code>auth/jwt.go</code>	30m	Silent auth context bug
P2	No indicator unit tests	<code>internal/indicators/</code>	2 days	Wrong signals on real money
P3	Split <code>internal/data</code> into domain packages	Refactor	2 days	Grows more coupled over time
P3	<code>internal/app</code> imports 10 packages	<code>app/app.go</code>	1 day	Composition root doing domain work
P3	Audit log goroutine-per-event	<code>audit/audit.go</code>	3h	Goroutine pile-up under load

Priority	Issue	File	Effort	If Ignored
P3	Volume stored as float64	db/models.go	30m	Floating-point accumulation
P3	Indicator drop counter mislabeled as wsDrops	indicators_manager.go	15m	Misleading monitoring
P3	ingest.go imports Zerodha SDK types	data/ingest.go	2h	Pipeline tied to one broker